

Министерство цифрового развития, связи и массовых коммуникаций РФ
федеральное государственное бюджетное образовательное учреждение высшего
образования
«Сибирский государственный университет телекоммуникаций и информатики»
(СибГУТИ)

Кафедра вычислительных систем
Допустить к защите
Зав. кафедрой к.т.н., доцент
_____ Перышкова Е.Н.

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА БАКАЛАВРА

Разработка бота для генерации музыки при помощи
методов машинного обучения

Пояснительная записка

Студент Горецкая Е.В.

Институт ИВТ Группа ИБ-022

Руководитель к.т.н. доцент Ракитский А.А.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	3
1 ИИ для генерации музыки.....	4
1.1 Обзор	4
1.2 Выбор Supo AI	6
1.3 Инструменты разработки	8
2 Разработка проекта.....	9
2.1 Разработка бота	9
2.2 Внедрение Supo AI	17
2.3 Тестирование бота.....	20
ЗАКЛЮЧЕНИЕ	25
ПЕРЕЧЕНЬ СОКРАЩЕНИЙ И ОБОЗНАЧЕНИЙ.....	26
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ.....	27
ПРИЛОЖЕНИЕ А	28

ВВЕДЕНИЕ

С каждым годом внедрения искусственного интеллекта происходят все чаще и происходит это в абсолютно различных сферах, в том числе и в музыке. Использование искусственного интеллекта в музыке помогает облегчить процесс написания песен для новичков и любителей, а также помогает создателям контента, таким как блоггеры, или же разработчикам игр и приложений, использовать фоновую музыку без проблем с авторскими правами.

Искусственный интеллект в создании креативных работ уже не является чем-то новым, однако, рассматривая существующие инструменты для генерации музыки, можно явно сказать, что они не могут быть доступны каждому из-за того что не все имеют интуитивно понятный интерфейс, в который можно включить и недостаточную гибкость, и сложности с настройками, или же требуют высоких вычислительных затрат.

В связи с этим было принято решение интегрировать оптимальную нейросеть SUNO AI [1] в понятную и знакомую для многих пользователей платформу Telegram.

Таким образом мы сможем расширить применение инновационных технологий, представляя пользователям удобный и понятный инструмент для создания музыкальных композиций на основе вводимых текстовых подсказок в виде описания, жанра и текста.

1 ИИ для генерации музыки

1.1 Обзор

Для данной работы было опробовано несколько нейронных сетей для генерации музыки, среди которых была выбрана Suno AI.

Ниже разберем основные рассматриваемые варианты.

Magenta.io

Magenta – это исследовательский проект от Google, запущенный некоторыми инженерами из Google Brain, включающий в себя такие модели как MusicVAE и NSynth для создания и преобразования аудио [6].

Примеры моделей скрытого пространства включают SketchRNN для эскизов, NSynth для музыкального тембра и теперь MusicVAE: иерархический рекуррентный вариационный автокодировщик для изучения скрытых пространств для музыкальных партитур.

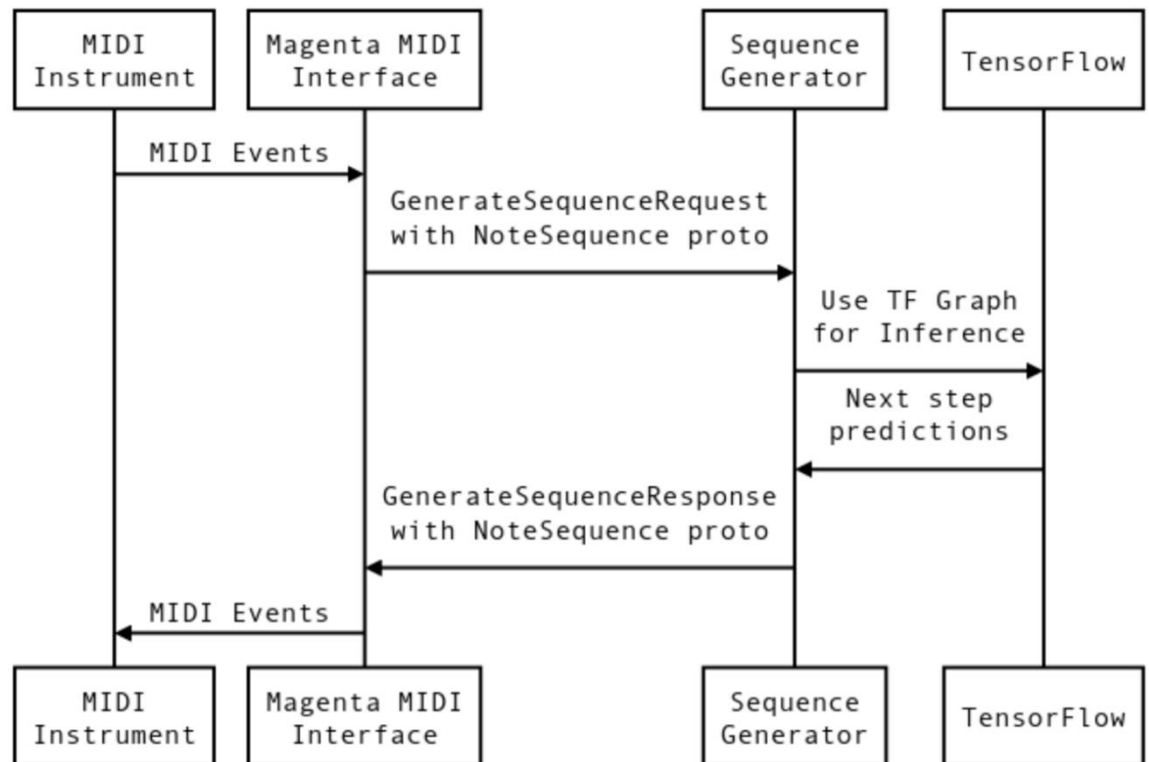


Рисунок 1. 1 - Magenta

Magenta предоставляет MIDI интерфейс для моделей TensorFlow. Создаются виртуальные MIDI порты для входа и выхода. Можно запускать несколько параллельных портов. К каждой паре портов можно подсоединить модель TensorFlow.

Преимущества:

1. большой выбор музыкальных инструментов и поддержка широкого спектра жанров
2. открытый код

Недостатки:

1. сложность интеграции с приложениями
2. неточная генерация вокальных партий

OpenAI Jukedeck

Описание: Jukedeck — это платформа для генерации музыки, разработанная компанией OpenAI. Она использует рекуррентные нейронные сети (RNN) для создания оригинальных музыкальных композиций на основе заданных параметров, таких как жанр, темп и настроение.

Преимущества

1. обучена генерировать музыку разнообразных стилей

Недостатки:

1. долгая генерация музыки
2. ограниченные возможности настройки инструментов
3. сгенерированная музыка зачастую звучит шаблонно

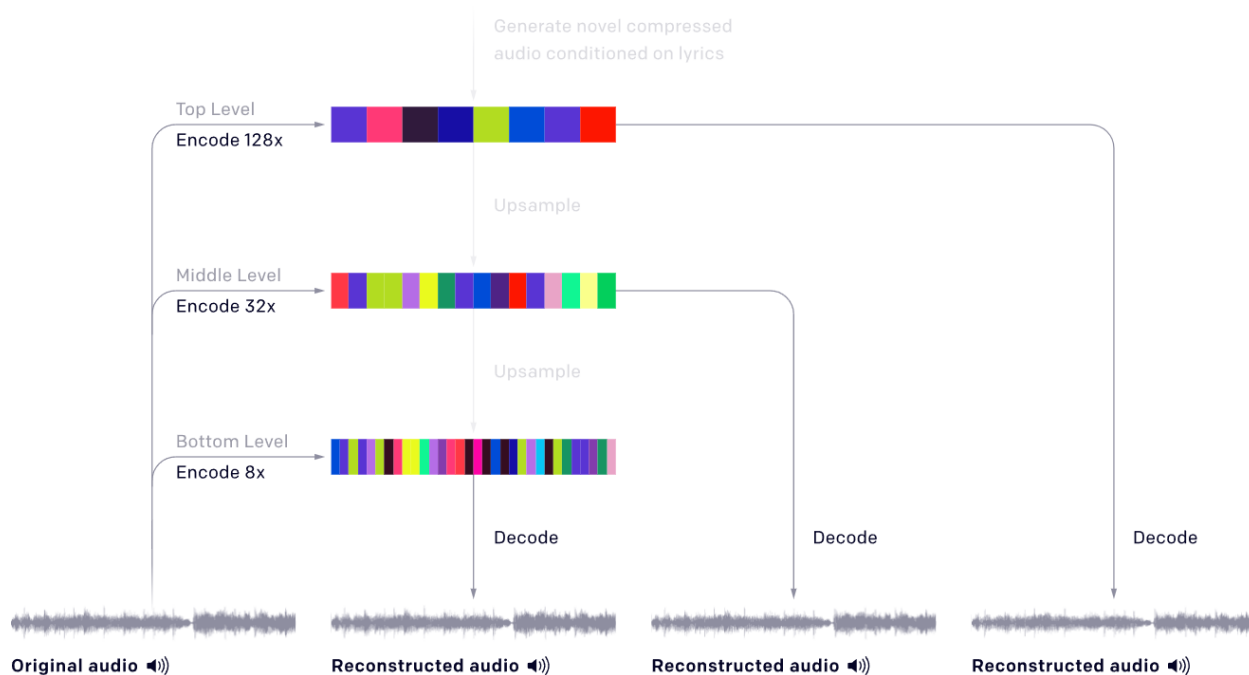


Рисунок 1. 2 - Jukedeck

Soundraw.io

Сервис позволяет создавать мелодии любого жанра. В Soundraw можно выбрать длительность (от 10 секунд до 5 минут), музыкальный стиль, настроение и темп. Есть встроенный микшер, который позволяет менять настройки сгенерированной композиции.

Преимущества:

1. для стриминговых платформ количество генераций треков не ограничивается
2. много опций

Недостатки:

1. для скачивания треков нужна платная подписка
2. ИИ создает однообразную музыку при одинаковых настройках, аудио получаются идентичными

1.2 Выбор Suno AI

Проведя тщательный анализ этих нейросетей выбор пал на SUNO AI.

Детали архитектуры закрыты, однако известно, что выбранная нейросеть базируется на двух основных моделях Bark и Chirp. Bark отвечает за вокал, а Chirp – за музыку.

Модель Bark - модель текст-в-аудио на основе трансформеров [3-5]. Она предназначена для различных задач генерации аудио. Её архитектурный стиль вдохновлен GPT и напоминает модели типа AudioLM и Vall-E. Bark использует квантованное аудиопредставление на основе EnCodec. В отличие от традиционных моделей текст-в-речь (TTS), Bark не ограничена линейным сценарием. Вместо этого она является полностью генеративной моделью текст-в-аудио с уникальной способностью создавать аудиоконтент, который может отклоняться от входного текста неожиданным образом. Эта особенность выделяет Bark среди обычных моделей TTS.

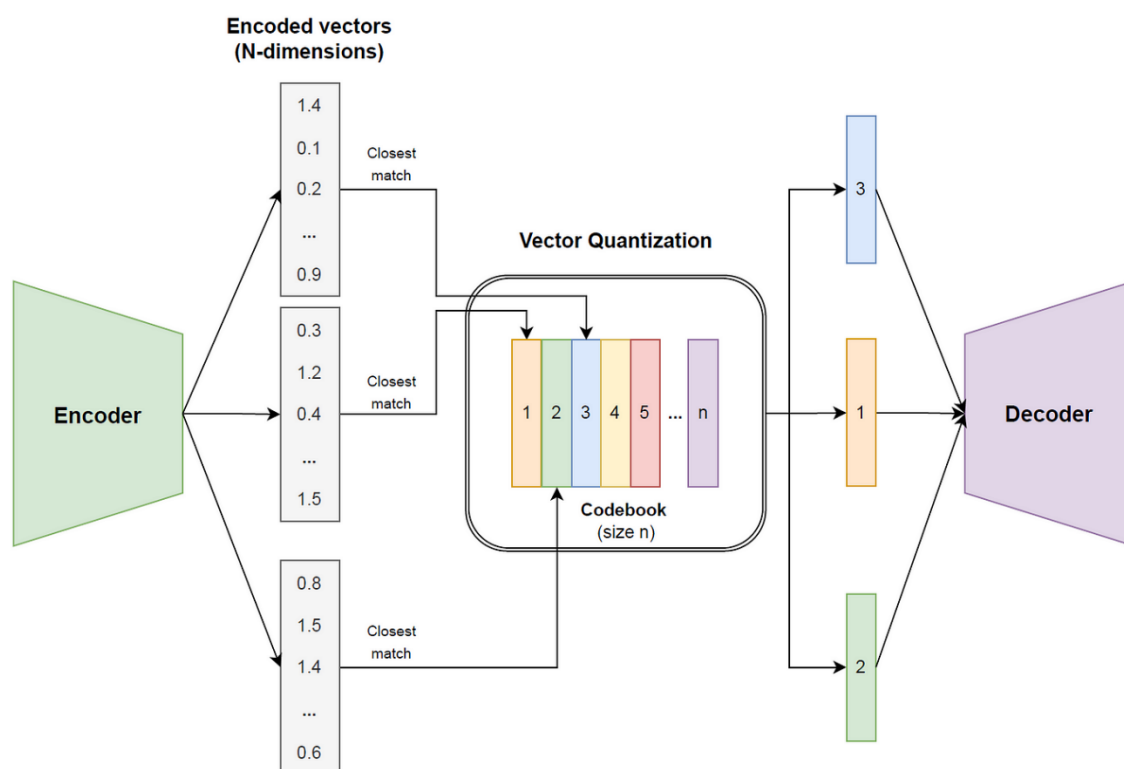


Рисунок 1. 3 - Bark

Bark специализируется на преобразовании текстового ввода в аудиовывод. Поддерживает большое количество языков. Имеет широкий спектр аудиоконтента, начиная от музыки и звуковых эффектов и заканчивая

фоновыми шумами и выражением эмоциональных всплесков, таких как смех, плач. Генерирует реалистичную речь и довольно четкое произношение на различных языках.

Chirp - это первоначальное название технологии преобразования текста в песню, созданной компанией Suno.

Также в Suno AI используются Flash Attention 2, который уменьшает задержку и повышает производительность при обработке больших объемов данных, и FP16 (полуточная точность), снижающая требования к памяти и ускоряет процесс инференса, что позволяет использовать модель на стандартных вычислительных ресурсах.

Подробная информация является закрытой, поэтому имеется только такое короткое описание.

1.3 Инструменты разработки

1. Язык разработки Python [2]
2. Среда разработки VS Code
3. Библиотека 'telebot' для работы с TelegramAPI
4. Для обеспечения взаимодействия были использованы функции из неофициального пакета SunoAPI

2 Разработка проекта

2.1 Разработка бота

Разработка проекта начинается с написания телеграм-бота, для этого мы подключили библиотеку `telebot`, взаимодействующую с TelegramAPI. В самом приложении Telegram, с помощью бота `botfather` создается бот, из которого берётся токен и копируется в наш код.

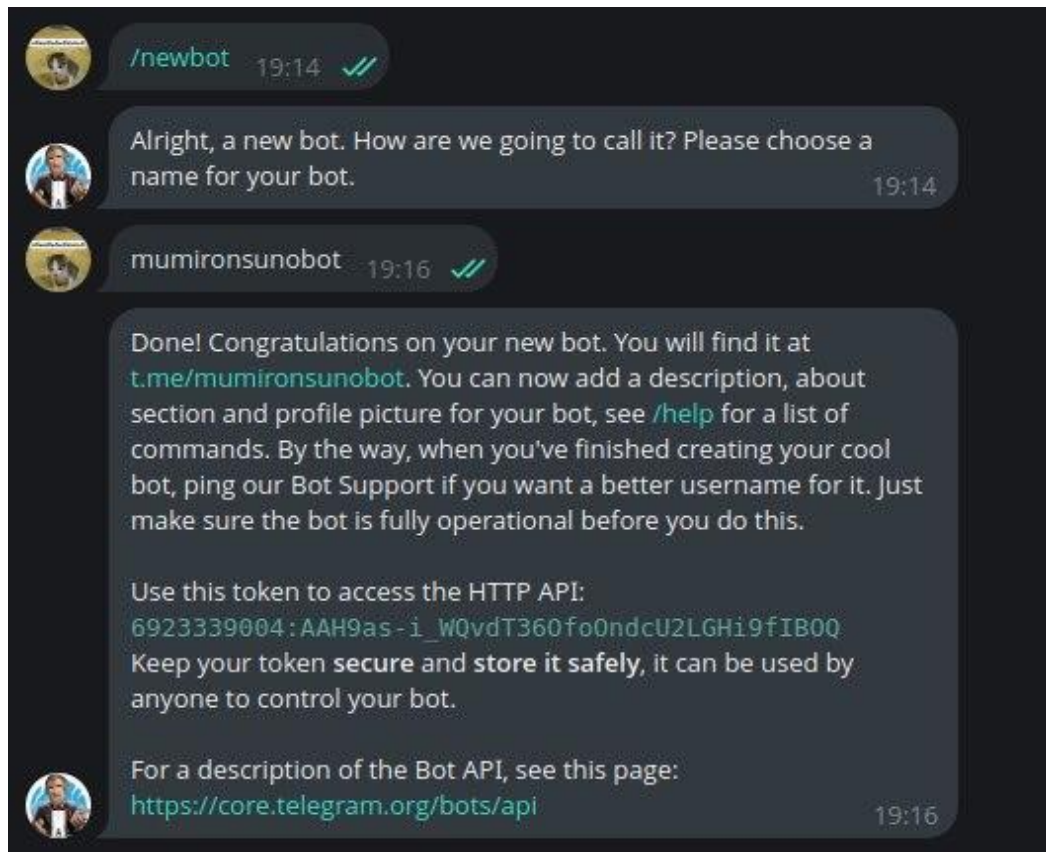


Рисунок 2. 1 Регистрация бота в botfather

Листинг 2. 1 – запись токена в переменную

```
bot = telebot.TeleBot('6923339004:AAH9as-  
i_WQvdT360foOndcU2LGHi9fIBOQ')
```

Пример работы с токеном можно увидеть в листинге 2.1.

После создали обработчик команд, который представляет из себя функцию, выводящую пользователю приветственное сообщение после

введения команды “/start”, и создающую кнопки “Create song” и “Help”. В случае выбора команды “Help” пользователю выводится вспомогательное сообщение. При выборе “Create song” – появляется выбор метода генерации музыки.

Функция обработки команд представлена в листинге 2.2

Листинг 2. 2 - Обработчик команд

```
@bot.message_handler(commands=['start'])
def send_welcome(message):
    markup = types.ReplyKeyboardMarkup(resize_keyboard=True,
row_width=2)
    btn1 = types.KeyboardButton('Create songs')
    btn2 = types.KeyboardButton('Help')
    markup.add(btn1, btn2)
    bot.send_message(message.chat.id, f"Hi,
{message.from_user.first_name}! Let's try creating music!",
reply_markup=markup)
```

В листинге 2.4 продемонстрирован обработчик пользовательских сообщений. Действия в этой функции определяются в зависимости от полученного сообщения.

Листинг 2. 3 - Вывод кнопок в функции create_song()

```
def create_song(message):
    markup = types.ReplyKeyboardMarkup(resize_keyboard=True,
row_width=2)
    btn1 = types.KeyboardButton('Fast create')
    btn2 = types.KeyboardButton('Custom create')
    btn3 = types.KeyboardButton('Back')
    markup.add(btn1, btn2, btn3)
    bot.send_message(message.chat.id, "How do you want to
create?", reply_markup=markup)
```

Функция `create_song()`, которую мы можем увидеть в листинге 2.3 создает кнопки и выводит сообщение пользователю.

Далее пользователь может выбрать “Fast create” для создания песни по текстовому описанию, “Custom create” - для генерации песни по 3 вводимым параметрам: текст, жанр и описание, “Back” – для возвращения в главное меню.

Листинг 2. 4 – Обработчик сообщений

```
@bot.message_handler(func=lambda message: True)
def handle_message(message):
    if message.text == 'Create songs':
        create_song(message)
    elif message.text == 'Fast create':
        bot.send_message(message.chat.id, "Send me the
description")
        bot.register_next_step_handler(message,
process_fast_create)
    elif message.text == 'Custom create':
        bot.send_message(message.chat.id, "Send me the lyrics
of the song")
        bot.register_next_step_handler(message,
process_custom_create_lyrics)
    elif message.text == 'Help':
        bot.send_message(message.chat.id, 'To create songs,
select a creation method and follow the instructions')
    elif message.text == 'Back':
        send_welcome(message)
```

Листинг 2.5 - Генерация по описанию

```
def process_fast_create(message):
    description = message.text
    bot.send_message(message.chat.id, f"You have entered:
{description}")
    bot.send_message(message.chat.id, "Songs are generated.
This may take 2-5 minutes")

    try:
        clips = client.songs.generate(
            prompt=description,
            instrumental=False,
        )
        bot.send_message(message.chat.id, "{0.first_name}, your
songs have been created! You can listen to
them:".format(message.from_user))
        bot.send_audio(message.chat.id,
audio=clips[0].audio_url)
        bot.send_audio(message.chat.id,
audio=clips[1].audio_url)
    except Exception as e:
        bot.send_message(message.chat.id, f"An error occurred:
{e}")
```

Для генерации музыки по описанию создана функция `process_fast_create()`, (представленный в листинге 2.5), возвращающая пользователю сообщение с введенным описанием и переходящая к методу `generate()`, в который передается текст пользователя.

Сам метод описан в файле `supo.py`, а рассмотрен в деталях будет ниже

Листинг 2. 6 - Фрагмент метода `generate()`

```
else:
    payload = {
        "mv": "chirp-v3-0",
        "prompt": "",
        "gpt_description_prompt": prompt,
        "make_instrumental": instrumental,
    }
```

“Custom create” использует несколько функций. В листинге 2.7 отображены следующие функции: `confirm_custom_create`: Эта функция обрабатывает ответ пользователя после подтверждения создания песни или запроса изменения параметров. Если пользователь согласен продолжить ('All right. Continue'), то вызывается метод `client.songs.generate` с параметрами для создания аудиоклипов.

Если пользователь хочет изменить жанр, тексты песни или описание, то функции для обновления параметров (`update_genre`, `update_lyrics`, `update_description`) регистрируются для следующего шага обработки.

`process_custom_create_genre`: Эта функция получает жанр песни (`genre`) от пользователя и переходит к следующему шагу — получению описания песни (`description`).

Листинг 2. 7 - Генерация по трем параметрам

```
def confirm_custom_create(message, lyrics, genre, description):
    if message.text == 'All right. Continue':
        bot.send_message(message.chat.id, "Songs are generated.
This may take 2-5 minutes")
        try:
            clips = client.songs.generate(
                lyrics,
                custom=True,
                tags=genre + ' ' + description,
                instrumental=False,
            )
            bot.send_message(message.chat.id, "{0.first_name},
your songs have been created! You can listen to
them:".format(message.from_user))
            bot.send_audio(message.chat.id,
audio=clips[0].audio_url)
            bot.send_audio(message.chat.id,
audio=clips[1].audio_url)
        except Exception as e:
            bot.send_message(message.chat.id, f"An error
occurred: {e}")
        elif message.text == 'Change genre':
            bot.send_message(message.chat.id, "Send me the new
genre")
            bot.register_next_step_handler(message, update_genre,
lyrics, description)
        elif message.text == 'Change lyrics':
            bot.send_message(message.chat.id, "Send me the new
lyrics")
            bot.register_next_step_handler(message, update_lyrics,
genre, description)
        elif message.text == 'Change description':
            bot.send_message(message.chat.id, "Send me the new
description")
            bot.register_next_step_handler(message,
update_description, lyrics, genre)
```

`process_custom_create_description`: Эта функция получает описание песни (`description`) и вызывает функцию `process_custom_create`, передавая все параметры (`lyrics`, `genre`, `description`).

Листинг 2. 8 - Получение параметров для custom create

```
def process_custom_create_lyrics(message):
    lyrics = message.text
    bot.send_message(message.chat.id, "Now send me the genre")
    bot.register_next_step_handler(message,
process_custom_create_genre, lyrics)

def process_custom_create_genre(message, lyrics):
    genre = message.text
    bot.send_message(message.chat.id, "Now send me the
description")
    bot.register_next_step_handler(message,
process_custom_create_description, lyrics, genre)

def process_custom_create_description(message, lyrics, genre):
    description = message.text
    #bot.send_message(message.chat.id, f"Lyrics:
{lyrics}\nGenre: {genre}\nDescription: {description}")
    process_custom_create(message, lyrics, genre, description)
```

`process_custom_create`: Эта функция выводит подтверждение всех параметров, введенных пользователем (тексты песни, жанр, описание), и предлагает пользователю выбрать, продолжить ли процесс или изменить какие-либо параметры. Для этого используется клавиатура с кнопками "All right. Continue", "Change genre", "Change lyrics", "Change description". После этого регистрируется следующий шаг обработки с помощью `bot.register_next_step_handler`.

Листинг 2. 9 - Подтверждение/изменение введенных параметров

```
def process_custom_create(message, lyrics, genre, description):
    bot.send_message(message.chat.id, f"Lyrics:
{lyrics}\nGenre: {genre}\nDescription: {description}")
    confirm_markup =
types.ReplyKeyboardMarkup(resize_keyboard=True, row_width=2)
    btn1 = types.KeyboardButton('All right. Continue')
    btn2 = types.KeyboardButton('Change genre')
    btn3 = types.KeyboardButton('Change lyrics')
    btn4 = types.KeyboardButton('Change description')
    confirm_markup.add(btn1, btn2, btn3, btn4)
    bot.send_message(message.chat.id, "All right, or do you
want to change something?", reply_markup=confirm_markup)
    bot.register_next_step_handler(message,
confirm_custom_create, lyrics, genre, description)
```

Функции `update_genre`, `update_lyrics`, `update_description` обновляют соответствующие параметры (жанр, тексты песни, описание) и вызывают функцию `process_custom_create`, чтобы пользователь мог подтвердить или изменить данные еще раз.

Листинг 2. 10 - Обновление измененных параметров

```
def update_genre(message, lyrics, description):
    genre = message.text
    process_custom_create(message, lyrics, genre, description)

def update_lyrics(message, genre, description):
    lyrics = message.text
    process_custom_create(message, lyrics, genre, description)

def update_description(message, lyrics, genre):
    description = message.text
    process_custom_create(message, lyrics, genre, description)
```


2.2 Внедрение Suno AI

Для соединения бота с Suno AI создается объект класса Suno, фрагмент которого представлен в листинге 2.11. В конструктор класса передаётся значение COOKIE выгруженное с сайта Suno.ai в процессе авторизации (см. рис. 2.2).

Листинг 2. 11 - Фрагмент кода класса Suno

```
class Suno(Client):

    def __init__(self, cookie: Optional[str] = None) -> None:
        if cookie is None:
            cookie = COOKIE
        if cookie == "":
            raise Exception("environment variable SUNO_COOKIE
is not set")
        super().__init__(cookie)
        self._sid = self._get_sid()
        self.songs = Songs(self)

    def _get_sid(self) -> str:
        url =
"https://clerk.suno.com/v1/client?_clerk_js_version=4.72.0"
        response = super().request("GET", url)
        if not response.ok:
            raise Exception(f"failed to get SID:
{response.status_code}")
        data = response.json()
        return
data.get("response").get("last_active_session_id")

. . .
```

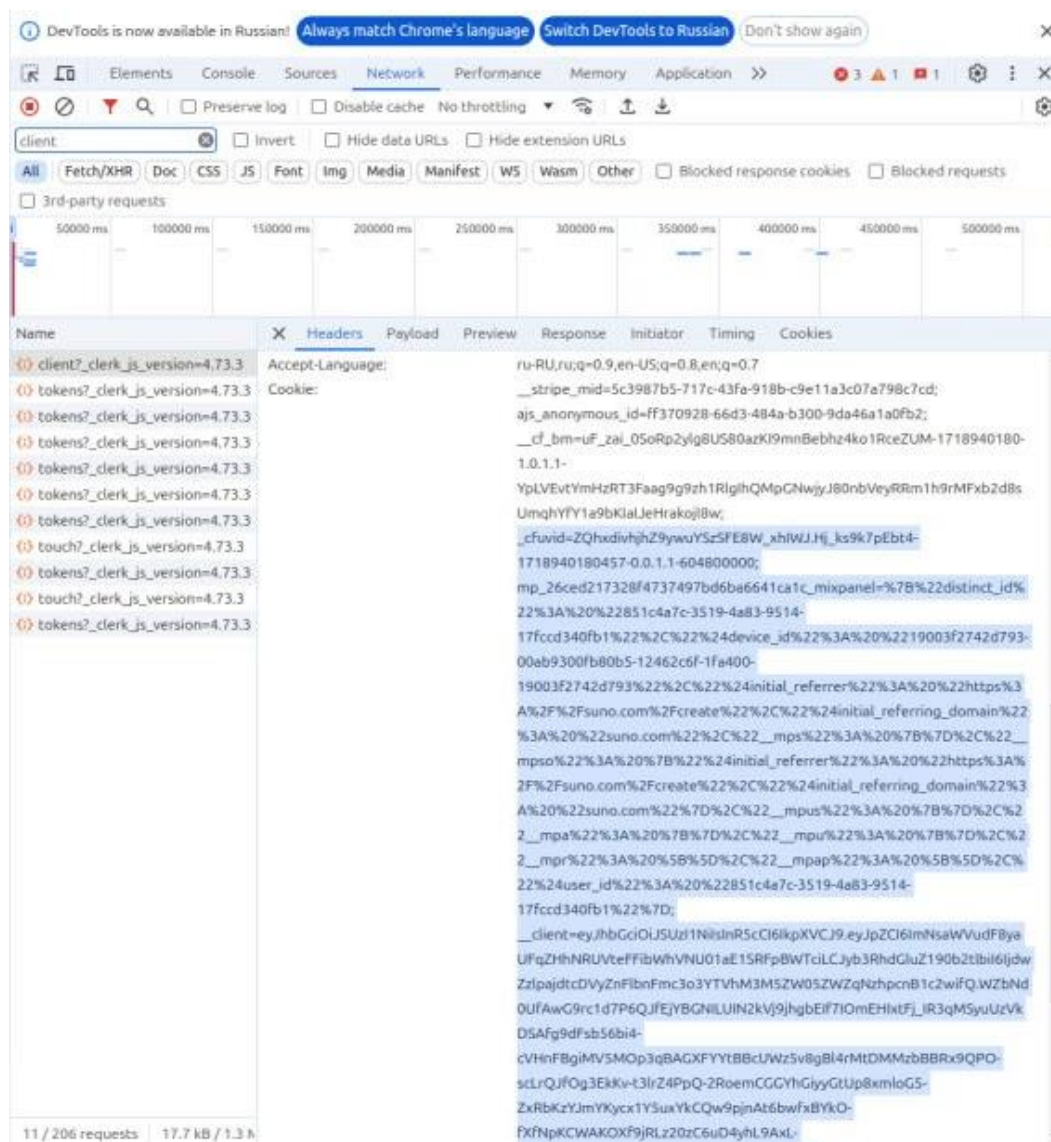


Рисунок 2. 2 Нахождение значения cookie

Для генерации композиции используется метод `generate`, представленный в листинге 2.12, этот метод за счет параметра `custom` определяет тип генерации (`fast` или `custom`). Остальные параметры определяют то, какая именно композиция и в каком жанре/стиле будет сгенерирована.

Листинг 2.12 – Метод `generate()`

```
def generate(
    self,
    prompt: str,
    custom: bool = False,
    tags: str = "",
```

```

        instrumental: bool = False,
    ) -> List[Song]:
        url = "https://studio-api.suno.ai/api/generate/v2/"
        timeout = 600
        # TODO: refactor this
        if custom:
            # create a song without lyrics
            if instrumental:
                payload = {
                    "mv": "chirp-v3-0",
                    "tags": prompt, # style of music
                    "prompt": "",
                    "make_instrumental": instrumental,
                }
            # create a song with lyrics
            else:
                payload = {
                    "mv": "chirp-v3-0",
                    "tags": tags, # style of music
                    "prompt": prompt, # lyrics
                    "make_instrumental": instrumental,
                }
        else:
            payload = {
                "mv": "chirp-v3-0",
                "prompt": "",
                "gpt_description_prompt": prompt,
                "make_instrumental": instrumental,
            }
        data = json.dumps(payload)
        response = self.request("POST", url, data=data)
        if not response.ok:
            raise Exception(f"failed to generate songs:
{response.status_code}")
        data = response.json()
        songs = data.get("clips")

```

```

ids = [song.get("id") for song in songs]
start = time.time()
while True:
    if time.time() - start > timeout:
        raise Exception("failed to generate songs: TIMEOUT")
    if sum([self._is_ready(id) for id in ids]) == len(ids):
        print("")
        break
    else:
        print(".", end="", flush=True)
for id in ids:
    print(f"song link: https://app.suno.ai/song/{id}")
return [self._get(id) for id in ids]

```

2.3 Тестирование бота

На рис.2.3 отображены запуск бота через команду “/start”, отображение кнопок “Create songs” и “Help”.

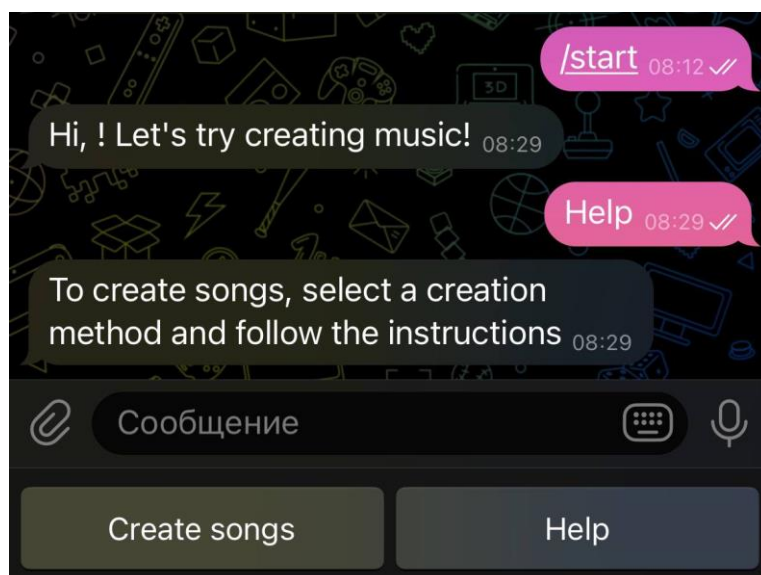


Рисунок 2. 3 – Запуск бота

При нажатии на кнопку “Create songs” пользователю задается вопрос и представляется выбор метода генерации музыки, а так же отображается кнопка, возвращающая в главное меню. (Рис.2.4)

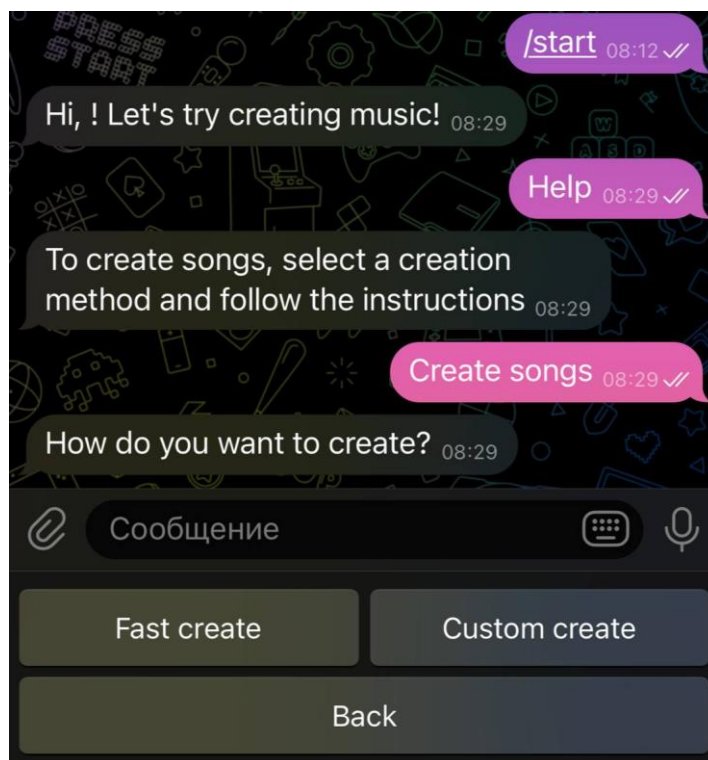


Рисунок 2. 4 – Смена кнопок при вызове команды Create songs

При выборе “Fast create” от пользователя требуется ввести описание песни (description), после ввода, как показано на рис.2.5, приходит сообщение от бота о времени ожидания генерации, а после - сообщение о готовности аудио и сами аудиофайлы соответственно.

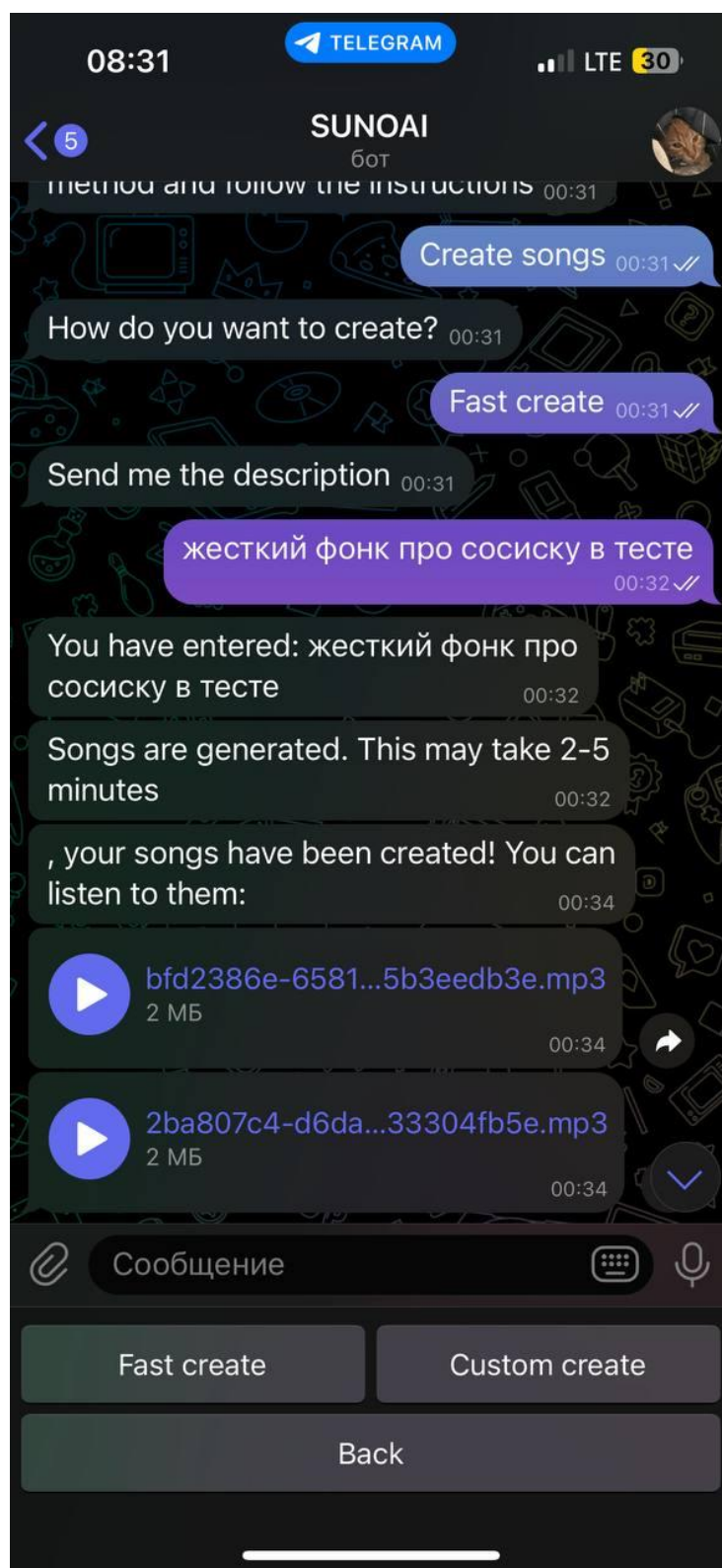


Рисунок 2. 5 – Генерация по описанию (Fast create)

При выборе метода “Custom create” от пользователя запрашиваются: текст (lyrics), жанр (genre), описание (description). После ввода приходит

сообщение с введенными данными и предлагается выбор редактирования каждого из пунктов, а также продолжение без изменений. (Рис. 2.6)

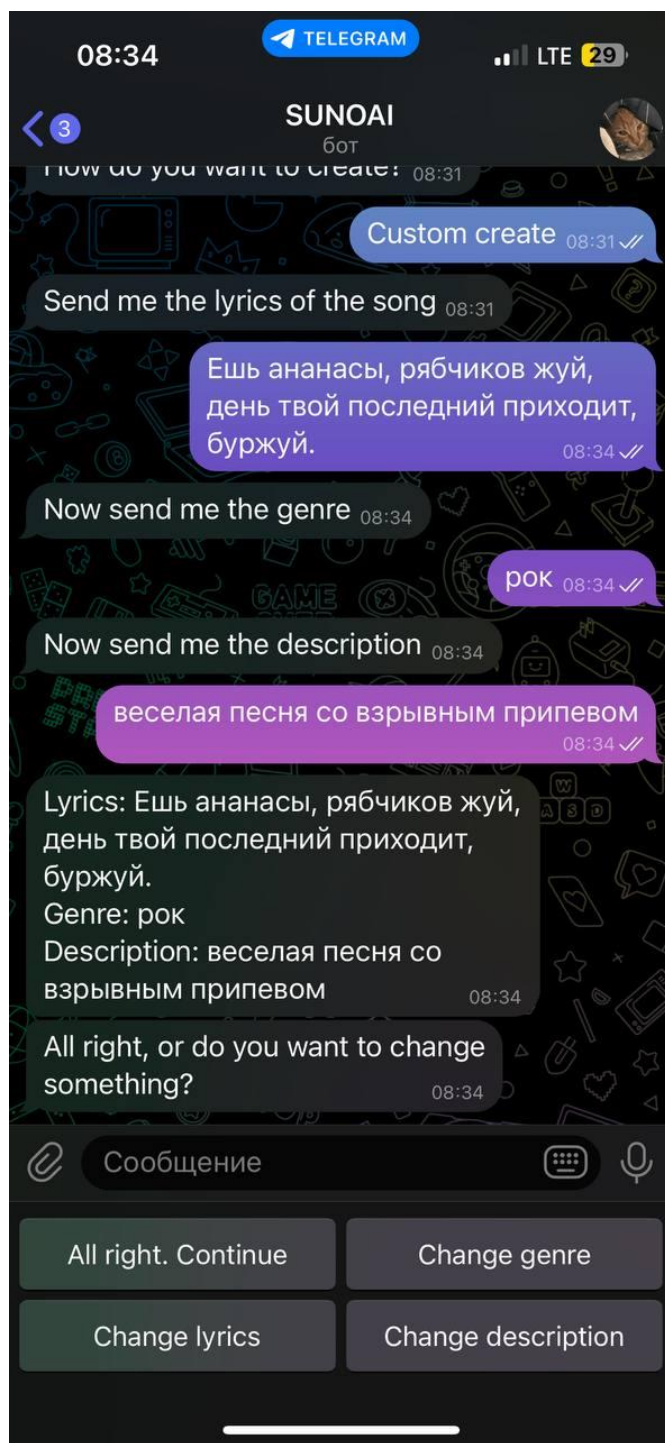


Рисунок 2. 6 – Генерация по трем параметрам (Custom create)

Измененные параметры обновляются и выводится полный обновленный список введенных данных. (Рис.2.7)

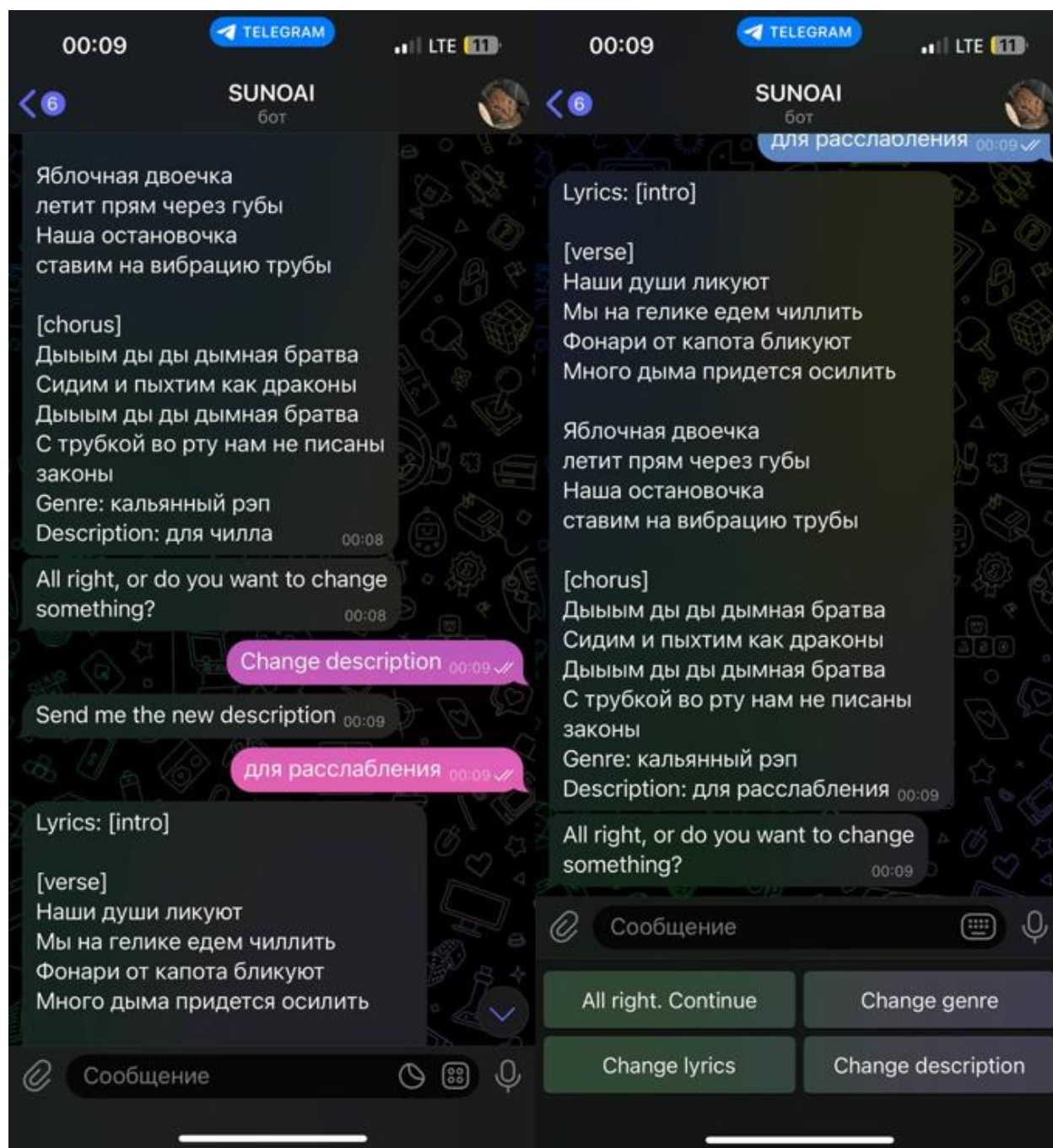


Рисунок 2. 7 – Редактирование введенных данных

ЗАКЛЮЧЕНИЕ

В ходе выполненной работы были изучены и проанализированы нейронные сети Magenta.io, OpenAI Jukedeck, Soundraw.io. В результате анализа и тестирования была выбрана наиболее подходящая для проекта SunoAI. Изучены и использованы библиотека telebot для взаимодействия с TelegramApi, а также отдельные методы из пакета SunoAPI, что позволило интегрировать SunoAI в функционал телеграм-бота. Проведена разработка и реализация телеграм-бота, аудиофайлы генерируются, все поставленные задачи выполнены.

ПЕРЕЧЕНЬ СОКРАЩЕНИЙ И ОБОЗНАЧЕНИЙ

В пояснительной записке применяются следующие сокращения и обозначения:

ИИ – искусственный интеллект

БР – бакалаврская работа

ВС – вычислительная система

ПЗ – пояснительная записка

ПМиК – прикладная математика и кибернетика

СибГУТИ – Сибирский государственный университет
телекоммуникаций и информатики

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

- 1 [Электронный ресурс] Suno AI. URL: <https://suno.com/> (Дата обращения 10.06.2024)
- 2 [Электронный ресурс] Python official documentation. URL: <https://www.python.org/doc/> (Дата обращения 10.06.2024)
- 3 Schumacher D., LaBounty Jr F., Schellekens M. Enhancing Suno's Bark Text-to-Speech Model: Addressing Limitations Through Meta's EnCodec and Pre-Trained Hubert //Available at SSRN 4443815. – 2023.
- 4 [Электронный ресурс] Medium. Text-to-Audio Generation with Bark, Clearly Explained URL: <https://betterprogramming.pub/text-to-audio-generation-with-bark-clearly-explained-4ee300a3713a> (Дата обращения 27.05.2024)
- 5 [Электронный ресурс] Github. BARK AI. URL: <https://github.com/serp-ai/bark-with-voice-clone> (Дата обращения 01.06.2024)
- 6 [Электронный ресурс] Habr. URL: <https://habr.com/ru/articles/342306/> (Дата обращения 01.06.2024)

ПРИЛОЖЕНИЕ А

```
import telebot
from telebot import types # для указание типов

import suno
client = suno.Suno(cookie =
"_cfuvid=iyXoIlRSZwwg7ic1_ziI.q5kZCDRifQhEUWxUiTjx4I-
1718381764161-0.0.1.1-604800000; __stripe_mid=5c3987b5-717c-
43fa-918b-c9e11a3c07a798c7cd; __stripe_sid=da6e00ce-12c8-4b2a-
84ea-bacbfef83fe7ca082c9;
__cf_bm=FWe9esnclxjxWTuy4kJp3ukG8CaGxRuuG1Ea5H13wdY-1718382671-
1.0.1.1-
KtqLpiHm5q6016vzQW2912iIWDazqPn9K8DWEikpbjAex0P5zaGMrdYo2dyZSttV
8fNjHw_0_hsyOw6sHOuXxG;
mp_26ced217328f4737497bd6ba6641calc_mixpanel=%7B%22distinct_id%2
2%3A%20%22851c4a7c-3519-4a83-9514-
17fccd340fb1%22%2C%22%24device_id%22%3A%20%2219003f2742d793-
00ab9300fb80b5-12462c6f-1fa400-
19003f2742d793%22%2C%22%24initial_referrer%22%3A%20%22https%3A%2
F%2Fsuno.com%2Fcreate%22%2C%22%24initial_referring_domain%22%3A%
20%22suno.com%22%2C%22__mps%22%3A%20%7B%7D%2C%22__mpso%22%3A%20%
7B%7D%2C%22__mpus%22%3A%20%7B%7D%2C%22__mpa%22%3A%20%7B%7D%2C%22
__mpu%22%3A%20%7B%7D%2C%22__mpr%22%3A%20%5B%5D%2C%22__mpap%22%3A
%20%5B%5D%2C%22%24user_id%22%3A%20%22851c4a7c-3519-4a83-9514-
17fccd340fb1%22%7D;
__client=eyJhbGciOiJSUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6ImNsaWVudF
8yaHNxb3ZxempuZExSRk5ValVueGRiT0Nwa3UiLCJyb3RhZGluZ190b2t1biI6Ij
FrdWtqcm9jbHhjbmszMnNsZGlscjBjYXU5bnpmY3l0YWl2NHl3dGcifQ.n7nJSxX
5ujFCXbha21XSfC0G9D4jypOw90lbwItsD7WV1Gv-9sbLniinp44jixUsuL---
GIcortD-5Sr4Ryu_a6ejPxNakgiYskxjvRNRLndE-B8UqcINLxoZ_g430k-
8vjAHUZGPygInTwz4F4uYCcaYVJCMuzxXEj4HP7DTwxV2imDF6aAdsX_ttXc7hIJ
7NM7mGXm5715cQW0md7Uj-5ql_PPi00sTG-
8AinlCL7bxT89UPcGgcTlCFXIOie6lHFQWxEdJ59_D6pyU5TijOjyNMcnpeE8Y3Jg
Lrs_OqoAidzt_l2xTWYwmMuBylniB78ZQmHb37LS3zNRnqUirpw;
__client_uat=1718383446")

bot = telebot.TeleBot('6923339004:AAH9as-
i_WQvdT36OfOondcU2LGHi9fIBOQ')

@bot.message_handler(commands=['start'])
def send_welcome(message):
    markup =
types.ReplyKeyboardMarkup(resize_keyboard=True, row_width=2)
    btn1 = types.KeyboardButton('Create songs')
    btn2 = types.KeyboardButton('Help')
    markup.add(btn1, btn2)
    bot.send_message(message.chat.id, f"Hi,
{message.from_user.first_name}! Let's try creating music!",
reply_markup=markup)
```

```

def create_song(message):
    markup =
types.ReplyKeyboardMarkup(resize_keyboard=True, row_width=2)
    btn1 = types.KeyboardButton('Fast create')
    btn2 = types.KeyboardButton('Custom create')
    btn3 = types.KeyboardButton('Back')
    markup.add(btn1, btn2, btn3)
    bot.send_message(message.chat.id, "How do you want to
create?", reply_markup=markup)

@bot.message_handler(func=lambda message: True)
def handle_message(message):
    if message.text == 'Create songs':
        create_song(message)
    elif message.text == 'Fast create':
        bot.send_message(message.chat.id, "Send me the
description")
        bot.register_next_step_handler(message,
process_fast_create)
    elif message.text == 'Custom create':
        bot.send_message(message.chat.id, "Send me the
lyrics of the song")
        bot.register_next_step_handler(message,
process_custom_create_lyrics)
    elif message.text == 'Help':
        bot.send_message(message.chat.id, 'To create songs,
select a creation method and follow the instructions')
    elif message.text == 'Back':
        send_welcome(message)

def process_fast_create(message):
    description = message.text
    bot.send_message(message.chat.id, f"You have entered:
{description}")
    bot.send_message(message.chat.id, "Songs are generated.
This may take 2-5 minutes")

    try:
        clips = client.songs.generate(
            prompt=description,
            instrumental=False,
        )
        bot.send_message(message.chat.id, "{0.first_name},
your songs have been created! You can listen to
them:".format(message.from_user))
        bot.send_audio(message.chat.id,
audio=clips[0].audio_url)
        bot.send_audio(message.chat.id,
audio=clips[1].audio_url)
    except Exception as e:
        bot.send_message(message.chat.id, f"An error
occurred: {e}")

```

```

def process_custom_create_lyrics(message):
    lyrics = message.text
    bot.send_message(message.chat.id, "Now send me the
genre")
    bot.register_next_step_handler(message,
process_custom_create_genre, lyrics)

def process_custom_create_genre(message, lyrics):
    genre = message.text
    bot.send_message(message.chat.id, "Now send me the
description")
    bot.register_next_step_handler(message,
process_custom_create_description, lyrics, genre)

def process_custom_create_description(message, lyrics,
genre):
    description = message.text
    #bot.send_message(message.chat.id, f"Lyrics:
{lyrics}\nGenre: {genre}\nDescription: {description}")
    process_custom_create(message, lyrics, genre,
description)

def process_custom_create(message, lyrics, genre,
description):
    bot.send_message(message.chat.id, f"Lyrics:
{lyrics}\nGenre: {genre}\nDescription: {description}")
    confirm_markup =
types.ReplyKeyboardMarkup(resize_keyboard=True, row_width=2)
    btn1 = types.KeyboardButton('All right. Continue')
    btn2 = types.KeyboardButton('Change genre')
    btn3 = types.KeyboardButton('Change lyrics')
    btn4 = types.KeyboardButton('Change description')
    confirm_markup.add(btn1, btn2, btn3, btn4)
    bot.send_message(message.chat.id, "All right, or do you
want to change something?", reply_markup=confirm_markup)
    bot.register_next_step_handler(message,
confirm_custom_create, lyrics, genre, description)

def confirm_custom_create(message, lyrics, genre,
description):
    if message.text == 'All right. Continue':
        bot.send_message(message.chat.id, "Songs are
generated. This may take 2-5 minutes")
        try:
            clips = client.songs.generate(
                lyrics,
                custom=True,
                tags=genre + ' ' + description,
                instrumental=False,
            )

```

```

        bot.send_message(message.chat.id,
"{0.first_name}, your songs have been created! You can listen to
them:".format(message.from_user))
        bot.send_audio(message.chat.id,
audio=clips[0].audio_url)
        bot.send_audio(message.chat.id,
audio=clips[1].audio_url)
    except Exception as e:
        bot.send_message(message.chat.id, f"An error
occurred: {e}")
    elif message.text == 'Change genre':
        bot.send_message(message.chat.id, "Send me the new
genre")
        bot.register_next_step_handler(message,
update_genre, lyrics, description)
    elif message.text == 'Change lyrics':
        bot.send_message(message.chat.id, "Send me the new
lyrics")
        bot.register_next_step_handler(message,
update_lyrics, genre, description)
    elif message.text == 'Change description':
        bot.send_message(message.chat.id, "Send me the new
description")
        bot.register_next_step_handler(message,
update_description, lyrics, genre)

    def update_genre(message, lyrics, description):
        genre = message.text
        process_custom_create(message, lyrics, genre,
description)

    def update_lyrics(message, genre, description):
        lyrics = message.text
        process_custom_create(message, lyrics, genre,
description)

    def update_description(message, lyrics, genre):
        description = message.text
        process_custom_create(message, lyrics, genre,
description)

    bot.polling()

```